

ActPlane: Letting Agents Harness Their Own System-Level Behavior with eBPF

Paper #XXX
Anonymous Submission

Abstract

AI coding agents require harnesses to enforce behavioral policies for effective and safe operation: e.g. do not leak secrets, run tests before committing. Our empirical study of 64 projects shows that while project harness instructions are declared in natural language, 63% consist of behavioral policies, 80% of which involve system-level behavior, and 78% of projects require cross-event state tracking.

Yet existing agent harnesses cannot connect intent-level declarations to deterministic system-level enforcement, leaving a *semantic gap*. Prompt constraints are probabilistic; tool-call guardrails miss subprocess behavior; OS-level enforcement is deterministic but returns opaque errors with no connection to declared intent. The policy engine should be exposed as a programmable interface that lets agents become the control plane of their own system-level actions. We realize this in ActPlane, which allows agents to declare cross-event behavior as labeled information-flow control (IFC) policies that observe and enforce the agents' actions across process, file, and network boundaries with semantic feedback. We implement ActPlane with eBPF and evaluate on the policies from the empirical study, showing that ActPlane can enforce complex cross-event policies with low overhead and provides actionable feedback to agents.

CCS Concepts: • **Computer systems organization** → *Embedded and cyber-physical systems*; • **Security and privacy** → *Software and application security*; • **Software and its engineering** → *Software safety*.

Keywords: AI agents, eBPF, BPF-LSM, labeled information-flow control, information-flow control, provenance, semantic feedback

ACM Reference Format:

Paper #XXX. 2026. ActPlane: Letting Agents Harness Their Own System-Level Behavior with eBPF. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26)*. ACM, New York, NY, USA, 7 pages.

1 Introduction

AI coding agents have become widely deployed tools for software development [12]. They operate as long-running processes with access to shells, source trees, package managers, and external APIs, autonomously writing code, running tests, and managing infrastructure across extended sessions.

As agents gain autonomy, they require harnesses to enforce behavioral policies. Projects encode such policies in

instruction files such as `CLAUDE.md` and `AGENTS.md` [5, 16]. Our empirical study of 64 projects extracts 2,152 statements and finds that 63% are behavioral directives, 80% of which involve system-level behavior, and 78% of projects require cross-event state tracking (§3). Because LLM compliance is inherently probabilistic [15], enforcement cannot reside inside the model [3, 25].

These policies require connecting intent-level declarations to system-level actions, yet existing approaches operate on only one side of this *semantic gap*. Intent-level guards [22, 26] lose coverage when agents shell out or spawn subprocesses: the same `git push` can be reached through a tool call, a `bash -c` string, or a compiled binary. OS-level enforcement [6, 14, 18] is unbypassable but requires policies to be statically pre-written by an administrator external to the agent, and returns opaque errors with no connection to declared intent. The agent—the entity that knows its own goals and must recover from enforcement—has no programmatic interface to define or evolve the constraints governing its own system-level actions.

Our insight is that the policy engine should be exposed as a programmable interface that lets agents become the control plane of their own system-level actions.

ActPlane is such an interface. Agents and developers declare behavioral policies in a compact DSL; ActPlane compiles them into labeled information-flow rules and loads them into an eBPF/BPF-LSM backend. Named labels propagate across process, file, and network objects at kernel hooks, encoding execution history as checkable state: when a process reads a file labeled `SECRET`, the process acquires that label; when it forks, the child inherits it. When a labeled subject matches a rule, ActPlane returns semantic feedback—which rule was matched, why, and what alternative path satisfies it. Each rule independently selects notify (observe), block (pre-operation denial), or kill (post-operation termination).

This paper makes three contributions.

1. It defines the *semantic gap* for AI agents and grounds it with an empirical study of instruction files from 64 popular projects, showing that cross-event information-flow policies are pervasive in real agent instruction files.

2. It presents ActPlane, a programmable OS-level control plane for agent harnesses. ActPlane compiles behavioral policies into eBPF/BPF-LSM labeled information-flow rules, enforces them across process, file, and network boundaries, and returns semantic feedback that reconnects rule matches to intent-level remediation.
3. It evaluates ActPlane on DSL expressiveness over 580 OS-level directives from the empirical study, agent translation correctness, system correctness across bypass paths, per-event overhead, and feedback effectiveness on Terminal-Bench tasks.

2 Background and Motivation

2.1 Agent Harnesses and Behavioral Policies

An agent harness is the non-model infrastructure that mediates between an LLM agent and its execution environment [3, 25]. Harness components include tool dispatch, memory management, orchestration, observation, enforcement, and feedback. This paper focuses on the observation, enforcement, and feedback components: the mechanisms that ensure an agent’s behavioral policies hold at runtime.

Behavioral policies are constraints that govern an agent’s observable effects rather than its internal reasoning. Projects encode them in instruction files such as CLAUDE.md and AGENTS.md [5, 16]: do not push directly to main; run tests before committing; never expose secrets to the network. These policies differ from coding-style directives (e.g., “prefer const over let”) in that they produce observable effects at the system-call boundary.

2.2 The Semantic Gap

An agent’s execution spans three abstraction levels.

Intent is the agent’s own goals and self-constraints, i.e., what it declares it will and will not do. **Action** is the tool calls the agent issues through its runtime (`read_file`, `run_command`, `edit_file`), mediated by the agent framework.

Behavior is the actual OS-level operations that result: `open`, `execve`, `connect`, and `fork`.

The mapping from action to behavior is many-to-many and opaque. A single `run_command("make")` triggers hundreds of system calls. The same `git push` can be reached through a direct tool call, a `bash -c` string, a Python subprocess, or a compiled helper binary. This opacity creates a *semantic gap*: policies declared at the intent level cannot be reliably enforced at a single intermediate level.

Enforcement placed at the intent level (prompt instructions) is probabilistic: LLM compliance degrades with context length and task complexity [15]. Enforcement placed at the action level (tool-call guards [22, 26]) is deterministic but bypassable: it intercepts tool calls, not the OS behavior they produce. Enforcement placed at the behavior level (syscall filters, sandboxes [6, 18]) is complete but returns opaque `-EPERM` failures with no connection to the agent’s declared

policies. Each level addresses a different part of the stack; none spans the full gap from intent to behavior.

A cooperative agent does not intentionally bypass tool-layer guards, but the opacity of the action-to-behavior mapping causes unintentional bypass in routine operation. Three representative scenarios illustrate this. (1) An agent calls `run_command("npm test")`; the test harness internally invokes `git push` to a staging server. The tool-call guard sees `npm test` (a permitted action), while the kernel observes the forbidden `git push` behavior. (2) An agent calls `read_file(".env")` to inspect a configuration value, then later calls `run_command("curl api.example.com")`. Each tool call is individually permitted, but the combination constitutes secret exfiltration, a cross-event condition visible only to an enforcer that tracks information flow from file read to network connect. (3) An agent writes a Python script containing `subprocess.run(["git", "push", "-force"])` and executes it. The tool-call guard sees `run_command("python script.py")`; the kernel observes the nested `execve("git", ["push", "-force"])`.

2.3 Threat Model

ActPlane targets a *cooperative but probabilistic* agent. The agent is not adversarial: it does not attempt to defeat the enforcement mechanism through obfuscation or exploitation. However, because LLM inference is inherently stochastic, the agent may trigger any given policy rule with non-zero probability at each step, regardless of context length or prompt design. Over a long session with many steps, the cumulative probability of at least one rule match approaches certainty.

The trust boundary is the kernel. The eBPF programs and BPF maps that propagate labels and check rules run in kernel space and cannot be modified by the agent. The policy DSL and feedback formatting run in a trusted userspace process (`actplane run`) that is not part of the agent’s process tree. The monitored domain is the agent’s entire process tree, including all descendants spawned through tools, shells, or subprocesses.

ActPlane does not defend against an agent that deliberately exploits kernel vulnerabilities or manipulates the eBPF loader. Such attacks require `CAP_BPF` or root privileges that the agent does not hold. Side-channel inference (deducing policy rules from `-EPERM` patterns), deliberate label pollution (triggering over-tainting to cause denial of service), and feedback-oracle attacks (using remediation strings to restructure an approach toward a forbidden goal through compliant steps) all require adversarial intent and are outside this threat model.

3 Empirical Study

This section characterizes behavioral policies in real agent instruction files and identifies the subset that requires cross-event information-flow observation and enforcement.

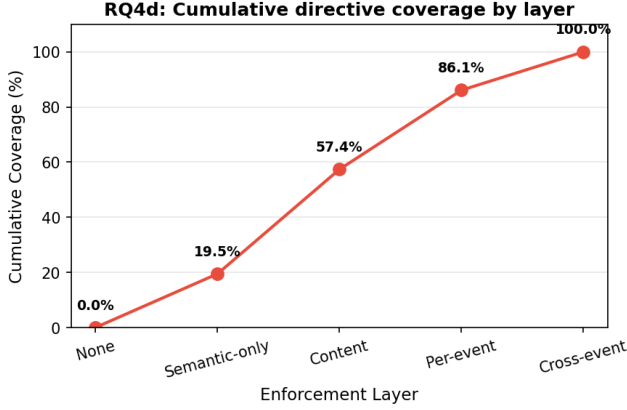


Figure 1. Cumulative directive coverage by enforcement layer. Linters cover 57.4%; per-event matching reaches 86.1%; cross-event IFC closes the remaining 13.9%.

3.1 Methodology

We collected CLAUDE.md and AGENTS.md files from 64 popular open-source repositories, extracted 2,152 statements, and classified each along three axes: content type (description vs. directive), topic (12 categories), and enforcement level (semantic-only, content, per-event, cross-event). Enforcement level is assigned by a waterfall decision procedure: each directive exits at the first level whose predicate it satisfies.

3.2 Findings

Instruction files are behavioral policies. 63% of statements are directives; 80.5% of directives involve system-observable behavior (commands, files, network).

Enforcement is layered. Content inspection covers 37.9% of directives, per-event syscall matching covers 28.7%, and 13.9% require cross-event state tracking (Figure 1).

Cross-event policies are pervasive at the project level. Although cross-event directives are 13.9% of all directives, 78% of repositories contain at least one, and 48% require all four enforcement layers (Figure 2).

3.3 Implications

Per-event policies can be enforced by matching a single system call. Cross-event policies cannot: they require tracking state across multiple operations. Enforcing them requires a mechanism that (1) propagates provenance across process, file, and network boundaries, (2) encodes it as checkable state, and (3) connects rule matches back to declared intent for semantic feedback. Labeled information-flow control satisfies all three requirements.

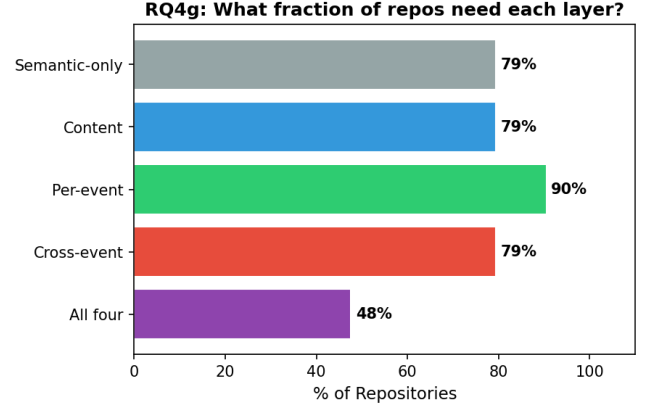


Figure 2. Fraction of repositories requiring each enforcement layer. 78% need cross-event tracking; 48% need all four layers.

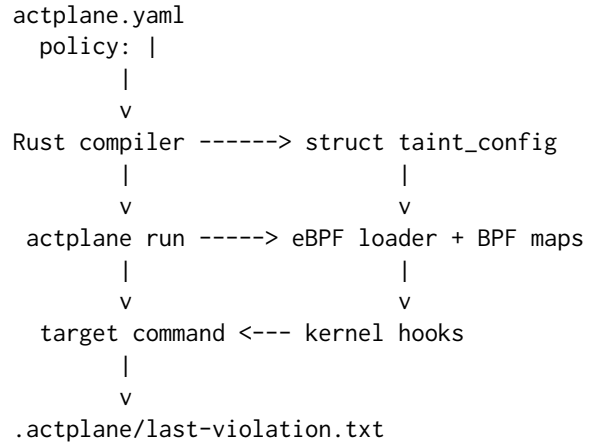


Figure 3. ActPlane pipeline. The Rust compiler lowers the policy DSL into a fixed-size binary configuration. The eBPF loader writes it into read-only data, attaches to kernel hooks, and reports rule matches as NDJSON. The runner maps rule identifiers back to human-readable feedback.

4 Design

ActPlane is a programmable OS-level control plane for agent harnesses. It compiles behavioral policies into labeled information-flow rules at OS kernel hooks, observing and enforcing agent behavior and returning semantic feedback when a rule is matched. Figure 3 summarizes the pipeline.

4.1 Policy Model

ActPlane policies are object-level information-flow rules over three node types: processes (identified by pid and command name), files (keyed by path hash), and endpoints (keyed by IPv4 address and mask). The model operates at object granularity rather than byte granularity [13], trading precision for low-overhead whole-system coverage.

```

source AGENT = exec "claude"
source SECRET = file "**/*.env"

# Per-event: from CoplayDev/unity-mcp
rule no-push-main:
  kill exec "git" "push" "main" if AGENT
  because "Do not push to main; open a PR instead."

# Cross-event: from chenhg5/cc-connect
rule tests-before-commit:
  block exec "git" "commit"
  if AGENT unless after exec "go" since write "**/*.go"
  because "Run `go test ./...` before committing."

```

Figure 4. Two ActPlane rules drawn from real projects. The first is a per-event rule (single `execve` match); the second is a cross-event rule requiring temporal state (after. . . since).

4.1.1 Labels and Propagation. Each node carries a 64-bit label bitmask. A *source* declaration introduces labels: an `exec` source labels processes that execute a matching binary, a `file` source labels files matching a path pattern, and an `endpoint` source labels network endpoints matching an address range. Let $L(x)$ denote the label bitmask of node x . Labels propagate through fixed transfer functions:

- **fork**($p \rightarrow c$): $L(c) := L(c) \cup L(p)$.
- **exec**(p, f): $L(p) := L(p) \cup L(f) \cup S(f)$, where $S(f)$ is the set of source labels matching f .
- **read**(p, f): $L(p) := L(p) \cup L(f)$.
- **write**(p, f): $L(f) := L(f) \cup L(p)$.
- **connect**(p, e): $L(e) := L(e) \cup L(p)$.

All transfer functions are monotonic: labels accumulate and are never removed by propagation. The only mechanism that removes a label is an explicit *declassify* directive in the DSL, which clears specified labels when the agent executes a designated gate binary (e.g., a redaction tool). This monotonicity bounds over-tainting: a node’s label set grows at most to the union of all source labels reachable through observed flow edges.

These transfer functions maintain a provenance invariant: if a process p carries label ℓ , then information from some source tagged with ℓ has reached p through observed OS-level flow edges.

4.1.2 Rules and Effects. A rule contains one or more clauses. Each clause starts with an action verb (`notify`, `block`, or `kill`) followed by an operation (`exec`, `read`, `write`, `unlink`, `connect`, or `open`), a target pattern, optional positional arguments, a Boolean expression over required and forbidden labels, and an optional condition (target allow-list, lineage gate, or temporal after-gate).

Each clause declares its own effect that determines how the kernel responds to a match.

Notify: the kernel emits a match event and allows the operation to proceed.

Block: a BPF-LSM hook returns `-EPERM` before the operation commits. Block is available only when the bpf LSM is active in the running kernel; it is not silently downgraded.

Kill: the kernel sends `SIGKILL` to the matching task. From a BPF-LSM hook, kill also returns `-EPERM`; from a tracepoint, kill is post-operation termination.

When multiple rules match the same event, the strongest effect applies in the order `kill > block > notify`.

4.1.3 Policy Language Scope. The DSL is not a general-purpose mandatory access control policy. Its primitives map to recurring agent workflow patterns: “after tests,” “through a review tool,” “data derived from untrusted input,” and “read-only sub-agent.” The grammar is small so that policy authors can translate natural-language instructions into source/sink/gate shapes without security-policy expertise.

4.2 System Architecture

4.2.1 Compiler and Kernel ABI. The Rust compiler parses the DSL and lowers it into a C-compatible struct `taint_config`. Lowering allocates label bits, expands Boolean expressions into required and forbidden bitmasks, lowers glob patterns into exact, prefix, suffix, or wildcard match kinds, and compiles IPv4 patterns into network/mask pairs. Human-readable rule names and reason strings (including remediation guidance) remain in Rust metadata; the kernel receives only fixed-size rule tables and emits integer rule identifiers.

This split keeps the eBPF program verifier-friendly. The kernel program iterates over fixed-size arrays in read-only data using bounded loops, copies each rule entry into a stack-local variable before matching, and uses explicit index bounds checks for argument scanning and pattern comparison.

4.2.2 Kernel State. The eBPF backend maintains five BPF hash maps:

- `ts_proc`: `pid` $\rightarrow$ label bitmask and lineage-gate state.
- `ts_root`, `ts_sess`: root-process and session-level temporal (after) gate state.
- `ts_file`: path hash $\rightarrow$ label bitmask.
- `ts_endp`: IPv4 address $\rightarrow$ label bitmask.

BPF map entries are keyed by `tid` (one entry per Linux thread) for `ts_proc` and by path hash or IPv4 address for object maps. Updates use atomic BPF helper operations; concurrent reads from different CPUs observe a consistent label bitmask per entry.

Tracepoint programs handle process lifecycle events (`fork`, `exec`, `exit`) and post-operation file and connect events. BPF-LSM programs attach to `bprm_check_security`, `file_open`, and `socket_connect` for pre-operation enforcement. When

BPF-LSM is not active, the loader uses tracepoint mode and disables block-effect rules. In tracepoint mode, hooks fire after the operation commits; a connect tracepoint observes an already-established connection. This is why block requires BPF-LSM: only LSM hooks provide a pre-operation verdict. Tracepoint mode supports notify (post-operation observation) and kill (post-operation task termination).

The current implementation supports up to 32 sources, 32 rules, 16 transforms, and 16 gates per policy. Rule evaluation uses `bpf_loop` for bounded iteration, which the eBPF verifier accepts for policies exceeding 100 rules.

4.2.3 Unified Event Handling. Each hook normalizes kernel arguments into a common event structure: subject pid, object kind, access kind, backend mode, target string, optional argument vector, and optional IPv4 address. A shared handler resolves the subject’s label bitmask, applies source labels for read/connect events, evaluates all supported rules, emits at most one `TAINT_VIOLATION` event for the highest-priority matching rule, and then applies propagation updates.

4.3 Corrective Feedback

The kernel match event contains the rule identifier, effect, target string, pid, and flags indicating whether the operation was blocked or the task was killed. The Rust runner maps the rule identifier to the corresponding rule name and reason string, then appends a structured payload to `.actplane/last-violation.txt`.

Agent integration uses a post-tool hook. The `actplane` feedback-hook subcommand reads new bytes from the feedback file and returns them as `additionalContext` to compatible agent runtimes [12]. The hook is a read-only relay: it does not re-evaluate policy in userspace. The kernel is the sole authority for rule matches; userspace formats the result so the agent can select an alternative execution path.

5 Implementation

ActPlane is implemented as a Rust driver and an eBPF program pair. The system builds a single `actplane` binary that embeds the compiled eBPF loader, extracts it at runtime, compiles the project’s policy DSL, and runs a target command under the observation and enforcement harness.

5.1 Runner and Policy Discovery

The command `actplane run - <cmd>` discovers `actplane.yaml` by searching upward from the working directory. The policy YAML contains a policy: | DSL block. Before the target process begins executing, the loader seeds the stopped target’s pid with the `AGENT` label so that all descendants inherit the agent identity. The runner prepares the feedback file (`.actplane/last-violation.txt`) and a hook state file, exports their paths to the child environment, waits for the loader to report readiness, then resumes the target. This sequencing ensures that the first target operation is observed.

5.2 BPF Backends

The eBPF program attaches to process lifecycle tracepoints (fork, exec, exit), syscall tracepoints for file and connect events, and BPF-LSM hooks for pre-operation enforcement. At load time, the loader checks whether the running kernel has the bpf LSM active in `/sys/kernel/security/lsm`. If not, it falls back to tracepoint mode, in which the supported effects are notify and kill; block rules are disabled because there is no pre-operation verdict path.

5.3 Limitations

Path identity. File labels are keyed by FNV-1a path hash rather than inode/device pairs. Hard links and mount binds can cause the same file to carry different labels under different paths.

Endpoint identity. Connect matching uses numeric IPv4 addresses. Hostname, DNS, SNI, and HTTP-layer matching require userspace resolution or additional instrumentation.

Argument predicates. Positional argument predicates inspect arguments after exec via the tracepoint argument buffer. Pre-operation blocking of argument predicates requires an argument cache before `bprm_check_security`, which is not yet implemented.

Label precision. Labels propagate at object granularity. A process that reads one byte from a labeled file acquires the full label set. This can produce over-tainting relative to byte-level systems [13]. Section 6 measures the false-positive rate and validates that declassification and gate paths prevent unnecessary harness intervention.

6 Evaluation

All evaluation rules are drawn from the empirical study corpus (64 real projects, 580 OS-level directives). No synthesized or abstract rules. The evaluation answers five questions.

6.1 Experimental Setup

6.2 DSL Expressiveness (RQ1)

How many real-world OS-level directives can the ActPlane DSL express?

Of the 1,361 directives in the empirical study, 580 are OS-level (391 per-event + 189 cross-event). For each, we assess whether the DSL can express the directive. We report coverage by enforcement level and by cross-event pattern type (temporal ordering, cross-file consistency, conditional execution, data flow, lineage mediation).

6.3 Agent Translation Correctness (RQ2)

Can an LLM agent correctly translate natural-language directives into ActPlane DSL rules?

An LLM agent reads each directive together with the project’s instruction file, source tree, and the DSL reference,

and produces a candidate rule. One author reviews each output and classifies it as correct (TP), incorrect (FP), or missed (FN). We report precision and recall by enforcement level.

6.4 System Correctness and Bypass Coverage (RQ3)

Does ActPlane correctly match rule conditions, and does it catch bypass paths that tool-layer guards miss?

For each confirmed-correct rule from RQ2, an LLM agent generates a violation trace (tool calls that should trigger the rule) and a compliant trace (correct-path tool calls that should not). Each trace is executed under four systems: (1) per-call tool-layer guard, (2) sequence-aware tool-layer guard, (3) per-event eBPF (ActPlane with label propagation disabled), and (4) full ActPlane. We report match rate and false-positive rate per system, broken down by per-event vs. cross-event rules.

6.5 Overhead (RQ4)

What is the per-event and end-to-end cost of label propagation and rule checking?

We measure per-syscall latency (fork, exec, open, write, connect) under rule counts of 1, 10, 32, and 100, reporting p50 and p99 latencies. We replay recorded agent traces under each configuration to measure end-to-end overhead. We also report BPF map memory consumption as a function of rule count and active process count.

6.6 Feedback Effectiveness (RQ5)

Does ActPlane’s semantic feedback improve agent task completion?

We run 89 Terminal-Bench tasks under three conditions: (B1) no ActPlane, (B2) ActPlane enforcement without feedback (bare `-EPERM`), and (B3) ActPlane enforcement with semantic feedback. A strong model generates per-task DSL rules; a weaker model executes tasks. We report task completion rate, guided completion rate (fraction of rule matches after which the agent completes the task), and repeat-match rate.

7 Related Work

7.1 In-Kernel Provenance and Information Flow

PASS introduced provenance as a storage-layer primitive [17]. Hi-Fi and LPM used LSM hooks to capture trustworthy whole-system provenance [2, 20]. CamFlow extended this to a streaming provenance framework over processes, files, and sockets [19].

CamQuery is the closest prior system [18]. It executes provenance queries inline in the kernel, propagates labels across process, file, and network edges, and can deny operations that match a policy rule. ActPlane shares this label-propagation model but differs in three respects: it targets cooperative AI agents rather than adversarial intrusions,

compiles an agent-oriented source/sink DSL rather than general provenance queries, and returns semantic feedback to the matching actor. The implementation differs as well: ActPlane uses the eBPF/BPF-LSM substrate rather than a loadable kernel module.

TaintDroid, Dytan, libdft, and Panorama established dynamic taint tracking at runtime, binary-instrumentation, and emulation layers [7, 10, 13, 27]. ActPlane operates at object granularity rather than byte granularity, trading precision for low-overhead whole-system coverage.

7.2 eBPF and LSM Enforcement

BPF-LSM, originally proposed as KRSI, allows eBPF programs to attach to Linux security hooks and return allow/deny verdicts [23, 24]. ActPlane uses BPF-LSM as its pre-operation enforcement backend. Contemporary eBPF enforcement tools such as Tetragon, Tracee, and eBPF-PATROL provide per-event predicate matching and, in Tetragon’s case, a boolean lineage flag (`followChildren`) that propagates across fork/exec. These systems evaluate per-event predicates or single-channel lineage flags rather than cross-channel labeled information flow [1, 6, 11].

7.3 Agent Guardrails and Information-Flow Control

AgentSpec and Progent provide deterministic runtime enforcement over tool-call names, arguments, and framework actions [22, 26]. These systems enforce at the tool-call boundary; ActPlane complements them by enforcing below the tool API, where effects that bypass the framework are still visible.

FIDES and CaMeL apply typed information-flow control and capability separation within the agent loop or scaffolding [8, 9]. ActPlane applies the same style of information-flow reasoning at the OS boundary, where observation and enforcement persist across context windows and cannot be circumvented by subprocess indirection.

7.4 Agent Instruction Files

Recent empirical studies characterize `CLAUDE.md`, `AGENTS.md`, and related agent instruction files along dimensions of content taxonomy, maintenance practices, and effect on task efficiency [4, 5, 16, 21]. ActPlane’s empirical study (§3) asks a different question: which instructions constitute cross-event information-flow policies, and which of those can be enforced at the OS level?

8 Conclusion

ActPlane provides a programmable OS-level control plane for agent harnesses. It compiles agent-declared behavioral policies into eBPF/BPF-LSM labeled information-flow rules, propagates labels across process, file, and endpoint objects at kernel hooks, and returns kernel-detected rule matches as semantic feedback to the agent. By supporting notify,

pre-operation denial, and harness-level termination with structured remediation, ActPlane enables agents to complete tasks after policy rule matches rather than fail opaquely. ActPlane unifies labeled information-flow observation and enforcement with an agent-oriented policy language and a feedback loop on the eBPF/BPF-LSM substrate, bridging the intent–behavior gap that single-level harnesses leave open.

References

- [1] Aqua Security. 2026. Tracee: Linux Runtime Security and Forensics using eBPF. <https://github.com/aquasecurity/tracee>.
- [2] Adam Bates, Dave Tian, Kevin R. B. Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, DC, 319–334. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates>
- [3] Birgitta Boeckeler. 2026. Harness Engineering for Coding Agent Users. <https://martinfowler.com/articles/harness-engineering.html>.
- [4] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884. <https://arxiv.org/abs/2511.12884>
- [5] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. arXiv:2509.14744. <https://arxiv.org/abs/2509.14744>
- [6] Cilium Project. 2026. Tetragon: eBPF-based Security Observability and Runtime Enforcement. <https://tetragon.io/>.
- [7] James Clause, Wanchun Li, and Alessandro Orso. 2007. Dytan: A Generic Dynamic Taint Analysis Framework. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*. Association for Computing Machinery, London, United Kingdom, 196–206. doi:10.1145/1273463.1273490
- [8] Manuel Costa, Boris Koepf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. <https://arxiv.org/abs/2505.23643>
- [9] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. <https://arxiv.org/abs/2503.18813>
- [10] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2010. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. USENIX Association, Vancouver, Canada, 393–407. <https://www.usenix.org/conference/osdi10/taintdroid-information-flow-tracking-system-realtime-privacy-monitoring>
- [11] Sangam Ghimire, Nirjal Bhurtel, Roshan Sahani, and Sudan Jha. 2025. eBPF-PATROL: Protective Agent for Threat Recognition and Overreach Limitation using eBPF in Containerized and Virtualized Environments. arXiv:2511.18155. <https://arxiv.org/abs/2511.18155>
- [12] Yuxuan Jiang, Meng Xu, Yiwen Song, and Xinwen Fu. 2025. AgentSight: Bridging the Semantic Gap Between Agent Intent and Low-Level Actions. arXiv:2508.02736. <https://arxiv.org/abs/2508.02736>
- [13] Vasileios P. Kemerlis, Georgios Portokalidis, Kangkook Jee, and Angelos D. Keromytis. 2012. libdft: Practical Dynamic Data Flow Tracking for Commodity Systems. In *Proceedings of the 8th ACM SIGPLAN/SIGOPS Conference on Virtual Execution Environments*. Association for Computing Machinery, London, United Kingdom, 121–132. doi:10.1145/2151024.2151042
- [14] Maxwell Krohn, Alexander Yip, Micah Brodsky, Natan Cliffer, M. Frans Kaashoek, Eddie Kohler, and Robert Morris. 2007. Information Flow Control for Standard OS Abstractions. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*. ACM, 321–334.
- [15] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. In *Transactions of the Association for Computational Linguistics*, Vol. 12. 157–173.
- [16] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv:2601.20404. <https://arxiv.org/abs/2601.20404>
- [17] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-Aware Storage Systems. In *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. USENIX Association, Boston, MA, 43–56. <https://www.usenix.org/conference/2006-usenix-annual-technical-conference/provenance-aware-storage-systems>
- [18] Thomas F. J.-M. Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eysers, Jean Bacon, and Margo Seltzer. 2018. Runtime Analysis of Whole-System Provenance. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, Toronto, Canada, 1601–1616. doi:10.1145/3243734.3243776
- [19] Thomas F. J.-M. Pasquier, Jatinder Singh, David Eysers, and Jean Bacon. 2017. CamFlow: Managed Data-Sharing for Cloud Services. *IEEE Transactions on Cloud Computing* 5, 3 (2017), 472–484. doi:10.1109/TCC.2015.2489211
- [20] Devin J. Pohly, Stephen McLaughlin, Patrick McDaniel, and Kevin Butler. 2012. Hi-Fi: Collecting High-Fidelity Whole-System Provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference*. Association for Computing Machinery, Orlando, FL, 259–268. doi:10.1145/2420950.2420989
- [21] Helio Victor F. Santos, Vitor Costa, Joao Eduardo Montandon, and Marco Tulio Valente. 2025. Decoding the Configuration of AI Coding Agents: Insights from Claude Code Projects. arXiv:2511.09268. <https://arxiv.org/abs/2511.09268>
- [22] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703. <https://arxiv.org/abs/2504.11703>
- [23] KP Singh. 2019. Kernel Runtime Security Instrumentation. Linux Plumbers Conference. <https://lpc.events/event/4/contributions/454/>
- [24] The Linux Kernel Documentation. 2021. LSM BPF Programs. https://www.kernel.org/doc/html/v5.15/bpf/bpf_lsm.html
- [25] Vivek Trivedy. 2026. The Anatomy of an Agent Harness. <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>.
- [26] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666. <https://arxiv.org/abs/2503.18666>
- [27] Heng Yin, Dawn Song, Manuel Egele, Christopher Kruegel, and Engin Kirda. 2007. Panorama: Capturing System-wide Information Flow for Malware Detection and Analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security*. Association for Computing Machinery, Alexandria, VA, 116–127. doi:10.1145/1315245.1315261